LANGAGE D'INTERROGATION DES DONNEES LID

I. Requêtes simples
II. Fonctions mono-lignes
III. Fonctions de groupes
IV. Jointures
V. Opérateurs ensemblistes
VI. Sous-interrogations

I. REQUETES SIMPLES

SELECT [DISTINCT] {*, colonne [as] [alias], …}

FROM table

- SELECT : indique les colonnes à récupérer
- DISTINCT : supprime les doublons
- FROM : indique les tables recherchées

REGLES D'ECRITURE DES ORDRES SQL

- Indifférence entre majuscules et minuscules
- Ecriture sur plusieurs lignes
- Clauses sont généralement placées sur des lignes distinctes
- Tabulations et indentations permettent une meilleure lisibilité

SELECTION DE TOUTES LES COLONNES

SELECT * FROM departments

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
30	Purchasing	114	1700
40	Human Resources	203	2400
50	Shipping	121	1500
60	IT	103	1400
70	Public Relations	204	2700
80	Sales	145	2500
90	Executive	100	1700
100	Finance	108	1700
110	Accounting	205	1700
120	Treasury	-	1700
130	Corporate Tax	-	1700
140	Control And Credit	-	1700
150	Shareholder Services	-	1700
160	Benefits	-	1700
170	Manufacturing	-	1700
180	Construction	-	1700
190	Contracting	-	1700
200	Operations	-	1700
210	IT Support	-	1700
220	NOC	-	1700
230	IT Helpdesk	-	1700
240	Government Sales	-	1700
250	Retail Sales	-	1700
260	Recruiting	-	1700
270	Payroll	-	1700

SELECTION D'UNE OU PLUSIEURS COLONNES

```
SELECT   department_id, department_name
FROM  departments
```

DEPARTMENT_ID	DEPARTMENT_NAME
10	Administration
20	Marketing
30	Purchasing
40	Human Resources
50	Shipping
60	IT
70	Public Relations
80	Sales
90	Executive
100	Finance
110	Accounting
120	Treasury
130	Corporate Tax
140	Control And Credit
150	Shareholder Services
160	Benefits
170	Manufacturing
180	Construction
190	Contracting
200	Operations
210	IT Support
220	NOC
230	IT Helpdesk
240	Government Sales
250	Retail Sales
260	Recruiting
270	Payroll

EXPRESSIONS ARITHMETIQUES

- **Expression contenant des données de type NUMBER , DATE et des opérateurs arithmétiques**

Opérateur	Description
+	Addition
-	Soustraction
*	Multiplication
/	Division

UTILISATION DES OPERATEURS ARITHMETIQUES

SELECT job_id, job_title, min_salary, max_salary, max_salary - min_salary FROM jobs

JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY	MAX_SALARY-MIN_SALARY
AD_PRES	President	20000	40000	20000
AD_VP	Administration Vice President	15000	30000	15000
AD_ASST	Administration Assistant	3000	6000	3000
FI_MGR	Finance Manager	8200	16000	7800
FI_ACCOUNT	Accountant	4200	9000	4800
AC_MGR	Accounting Manager	8200	16000	7800
AC_ACCOUNT	Public Accountant	4200	9000	4800
SA_MAN	Sales Manager	10000	20000	10000
SA_REP	Sales Representative	6000	12000	6000
PU_MAN	Purchasing Manager	8000	15000	7000
PU_CLERK	Purchasing Clerk	2500	5500	3000
ST_MAN	Stock Manager	5500	8500	3000
ST_CLERK	Stock Clerk	2000	5000	3000
SH_CLERK	Shipping Clerk	2500	5500	3000
IT_PROG	Programmer	4000	10000	6000
MK_MAN	Marketing Manager	9000	15000	6000
MK_REP	Marketing Representative	4000	9000	5000
HR_REP	Human Resources Representative	4000	9000	5000
PR_REP	Public Relations Representative	4500	10500	6000

VALEUR NULL

- NULL représente une valeur non disponible
- NULL # zéro, espace ou chaîne vide

SELECT employee_id, first_name, last_name, commission_pct
From employees

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	COMMISSION_PCT
100	Steven	King	-
101	Neena	Kochhar	-
102	Lex	De Haan	-
103	Alexander	Hunold	-
104	Bruce	Ernst	-
105	David	Austin	-
106	Valli	Pataballa	-

VALEUR NULL DANS LES EXPRESSIONS ARITHMETIQUES

- Les expressions arithmétiques comportant une valeur NULL sont évaluées à NULL

```
SELECT  first_name, last_name, (1+commission_pct )*salary
From employees
```

FIRST_NAME	LAST_NAME	COMMISSION_PCT+SALARY
Adam	Fripp	-
Alana	Walsh	-
Alberto	Errazuriz	12000,3
Alexander	Hunold	-
Alexander	Khoo	-
Alexis	Bull	-
Allan	McEwen	9000,35
Alyssa	Hutton	8800,25
Amit	Banda	6200,1
Anthony	Cabrio	-

ALIAS DE COLONNE

- Renomme un en-tête de colonne
- Suit le nom de colonne
- Doit obligatoirement être inclus entre guillemets s'il contient des espaces, des caractères spéciaux ou si la casse est respectée.

```
SELECT  first_name as "prénom", last_name  as nom, salary as
salaire, commission_pct as " % commission ",
salary*(1+commission_pct) as "salaire majoré "
From employees
```

Prénom	NOM	SALAIRE	% Commission	Salaire Majoré
Adam	Fripp	8200	-	-
Alana	Walsh	3100	-	-
Alberto	Errazuriz	12000	,3	15600
Alexander	Hunold	9000	-	-
Alexander	Khoo	3100	-	-
Alexis	Bull	4100	-	-
Allan	McEwen	9000	,35	12150
Alyssa	Hutton	8800	,25	11000
Amit	Banda	6200	,1	6820

OPERATEUR DE CONCATENATION

- Concatène des colonnes ou des chaines de caractères
- Est représenté par le symbole ||
- La colonne résultante est une expression caractère

```
SELECT  employee_id, last_name  || ' ' || first_name
as "nom et prénom "
FROM employees
```

EMPLOYEE_ID	Nom Et Prénom
100	King Steven
101	Kochhar Neena
102	De Haan Lex
103	Hunold Alexander
104	Ernst Bruce
105	Austin David
106	Pataballa Valli
107	Lorentz Diana
108	Greenberg Nancy
109	Faviet Daniel

ELIMINATION DES DOUBLONS

- Ajouter le mot-clé DISTINCT

SELECT distinct department_id FROM employees

DEPARTMENT_ID
10
20
30
40
50
60
70
80
90
100
110
-

SELECTION DES LIGNES

- **Filtrer la sélection au moyen de la clause where**

  SELECT [DISTINCT] {*, column [as] [alias] , … }
  FROM table
  WHERE condition(s)

  SELECT last_name, job_id, department_id FROM employees
  WHERE department_id=30

LAST_NAME	JOB_ID	DEPARTMENT_ID
Raphaely	PU_MAN	30
Khoo	PU_CLERK	30
Baida	PU_CLERK	30
Tobias	PU_CLERK	30
Himuro	PU_CLERK	30
Colmenares	PU_CLERK	30

CHAINES DE CARACTERES ET DATES

- Les constantes chaînes de caractères et dates doivent être placées entre simples quotes.
- La recherche tient compte de la casse pour les chaînes et du format pour les dates
- Le format de date par défaut est 'DD-MM-YY'

```
SELECT last_name, first_name, job_id, department_id
FROM employees
WHERE  last_name ='Taylor'
```

LAST_NAME	FIRST_NAME	JOB_ID	DEPARTMENT_ID
Taylor	Jonathon	SA_REP	80
Taylor	Winston	SH_CLERK	50

OPERATEURS DE COMPARAISONS

OPERATEUR	DESCRIPTION
=	Egal à
<	Inférieur à
<=	Inférieur ou égal à
>	Supérieur à
>=	Supérieur ou égal à
<> Ou !=	Différent

AUTRES OPERATEURS DE COMPARAISONS

OPERATEUR	DESCRIPTION
BETWEEN val1 AND val2	val1 <= val <= val2
IN (v1,v2, ...vn)	ϵ {v1,v2, …vn}
Like	Ressemblance partielle de chaînes de caractètes
IS NULL	Correspond à une valeur NULL

UTILISATION DE L'OPERATEUR BETWEEN

```
SELECT last_name, salary
FROM employees
WHERE  salary BETWEEN 12000 AND 15000
```

LAST_NAME	SALARY
Greenberg	12000
Russell	14000
Partners	13500
Errazuriz	12000
Hartstein	13000
Higgins	12000

```
SELECT last_name, salary
FROM employees
WHERE  last_name  BETWEEN 'A'  AND  'C'
```

LAST_NAME	SALARY
Abel	11000
Ande	6400
Atkinson	2800
Austin	4800
Baer	10000
Baida	2900
Banda	6200
Bates	7300
Bell	4000
Bernstein	9500
Bissot	3300
Bloom	10000
Bull	4100

```
SELECT last_name, hire_date FROM employees
where hire_date between '01-01-87' and '31-12-90'
```

LAST_NAME	HIRE_DATE
King	17/06/87
Kochhar	21/09/89
Hunold	03/01/90
Whalen	17/09/87

UTILISATION DE L'OPERATEUR IN

- IN permet de comparer une expression avec une liste de valeurs

```
SELECT employee_id, last_name, salary, manager_id
FROM employees
WHERE  manager_id in (103, 201, 205)
```

EMPLOYEE_ID	LAST_NAME	SALARY	MANAGER_ID
104	Ernst	6000	103
105	Austin	4800	103
106	Pataballa	4800	103
107	Lorentz	4200	103
202	Fay	6000	201
206	Gietz	8300	205

```
SELECT employee_id, last_name, salary, manager_id
FROM employees  WHERE  department_id in (10,20)
```

EMPLOYEE_ID	LAST_NAME	SALARY	MANAGER_ID
200	Whalen	4400	101
201	Hartstein	13000	100
202	Fay	6000	201

UTILISATION DE L'OPERATEUR LIKE

- LIKE permet de rechercher des chaînes de caractères à l'aide de caractères génériques
- Les conditions de recherche peuvent contenir des caractères ou des nombres littéraux
- % représente Zéro ou plusieurs caractères
- _ représente un caractère

```
SELECT employee_id, last_name, salary, manager_id
FROM employees WHERE last_name LIKE 'A%'
```

EMPLOYEE_ID	LAST_NAME	SALARY	MANAGER_ID
174	Abel	11000	149
166	Ande	6400	147
130	Atkinson	2800	121
105	Austin	4800	103

UTILISATION DE L'OPERATEUR LIKE

SELECT * FROM employees WHERE last_name like'__n%'

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	-	-	90
103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	-	102	60
104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	-	103	60
127	James	Landry	JLANDRY	650.124.1334	14/01/99	ST_CLERK	2400	-	120	50
156	Janette	King	JKING	011.44.1345.429268	30/01/96	SA_REP	10000	,35	146	80
167	Amit	Banda	ABANDA	011.44.1346.729268	21/04/00	SA_REP	6200	,1	147	80
195	Vance	Jones	VJONES	650.501.4876	17/03/99	SH_CLERK	2800	-	123	50

SELECT * FROM employees
WHERE last_name LIKE 'A_B%' ESCAPE '\'

Liste des employés dont le nom commence par le caractère A suivi du caractère _ suivi du caractère B.

UTILISATION DE L'OPERATEUR IS NULL

SELECT * FROM employees WHERE Manager_id IS NULL

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	-	-	90

OPERATEURS LOGIQUES

OPERATEUR	DESCRIPTION
AND	Retourne TRUE si les deux conditions sont vraies
OR	Retourne TRUE si au moins une des conditions est vraie
NOT	Inverse la valeur de la condition TRUE si la conditon est fausse FALSE si la condition est vraie

UTILISATION DE L'OPERATEUR AND

SELECT * FROM employees WHERE salary >= 1500 AND job_id='PU_CLERK'

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
115	Alexander	Khoo	AKHOO	515.127.4562	18/05/95	PU_CLERK	3100	-	114	30
116	Shelli	Baida	SBAIDA	515.127.4563	24/12/97	PU_CLERK	2900	-	114	30
117	Sigal	Tobias	STOBIAS	515.127.4564	24/07/97	PU_CLERK	2800	-	114	30
118	Guy	Himuro	GHIMURO	515.127.4565	15/11/98	PU_CLERK	2600	-	114	30
119	Karen	Colmenares	KCOLMENA	515.127.4566	10/08/99	PU_CLERK	2500	-	114	30

TABLE DE VERITE AND

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

UTILISATION DE L'OPERATEUR OR

SELECT * FROM employees WHERE salary >= 12000 OR job_id='PU_CLERK'

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	-	-	90
101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	-	100	90
102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	-	100	90
108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	-	101	100
115	Alexander	Khoo	AKHOO	515.127.4562	18/05/95	PU_CLERK	3100	-	114	30
116	Shelli	Baida	SBAIDA	515.127.4563	24/12/97	PU_CLERK	2900	-	114	30
117	Sigal	Tobias	STOBIAS	515.127.4564	24/07/97	PU_CLERK	2800	-	114	30
118	Guy	Himuro	GHIMURO	515.127.4565	15/11/98	PU_CLERK	2600	-	114	30
119	Karen	Colmenares	KCOLMENA	515.127.4566	10/08/99	PU_CLERK	2500	-	114	30
145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	,4	100	80
146	Karen	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	,3	100	80
147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	,3	100	80
201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	-	100	20
205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	-	101	110

TABLE DE VERITE OR

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

UTILISATION DE L'OPERATEUR NOT

```
SELECT * FROM employees
WHERE job_id NOT IN ('PU_CLERK', 'SA_MAN', 'SA_REP',
'SA_MAN', 'SH_CLERK', 'ST_CLERK')
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	-	-	90
101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	-	100	90
102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	-	100	90
103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	-	102	60
104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	-	103	60
105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	-	103	60
106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	-	103	60
107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	-	103	60
108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	-	101	100
109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	-	108	100
110	John	Chen	JCHEN	515.124.4269	28/09/97	FI_ACCOUNT	8200	-	108	100
111	Ismael	Sciarra	ISCIARRA	515.124.4369	30/09/97	FI_ACCOUNT	7700	-	108	100
112	Jose Manuel	Urman	JMURMAN	515.124.4469	07/03/98	FI_ACCOUNT	7800	-	108	100
113	Luis	Popp	LPOPP	515.124.4567	07/12/99	FI_ACCOUNT	6900	-	108	100
114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	-	100	30
120	Matthew	Weiss	MWEISS	650.123.1234	18/07/96	ST_MAN	8000	-	100	50
121	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	-	100	50
122	Payam	Kaufling	PKAUFLIN	650.123.3234	01/05/95	ST_MAN	7900	-	100	50
123	Shanta	Vollman	SVOLLMAN	650.123.4234	10/10/97	ST_MAN	6500	-	100	50
124	Kevin	Mourgos	KMOURGOS	650.123.5234	16/11/99	ST_MAN	5800	-	100	50
200	Jennifer	Whalen	JWHALEN	515.123.4444	17/09/87	AD_ASST	4400	-	101	10
201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	-	100	20
202	Pat	Fay	PFAY	603.123.6666	17/08/97	MK_REP	6000	-	201	20
203	Susan	Mavris	SMAVRIS	515.123.7777	07/06/94	HR_REP	6500	-	101	40

TABLE DE VERITE NOT

	TRUE	FALSE	UNKNOWN
NOT	FALSE	TRUE	UNKNOWN

REGLES DE PRIORITE

ORDRE DE PRIORITE	OPERATEUR
1	Les parenthèses
2	Tous les opérateurs de comparaison
3	NOT
4	AND
5	OR

REGLES DE PRIORITE

```
SELECT last_name, first_name, job_id, salary
FROM employees
WHERE  job_id='SA_MAN' OR job_id='AD_PRES' AND
salary > 15000
```

LAST_NAME	FIRST_NAME	JOB_ID	SALARY
King	Steven	AD_PRES	24000
Russell	John	SA_MAN	14000
Partners	Karen	SA_MAN	13500
Errazuriz	Alberto	SA_MAN	12000
Cambrault	Gerald	SA_MAN	11000
Zlotkey	Eleni	SA_MAN	10500

REGLES DE PRIORITE

```
SELECT last_name, first_name, job_id, salary
FROM employees
WHERE  (job_id='SA_MAN' OR job_id='AD_PRES' ) AND
salary > 15000
```

LAST_NAME	FIRST_NAME	JOB_ID	SALARY
King	Steven	AD_PRES	24000

TRI : CLAUSE ORDER BY

Tri des lignes avec la clause ORDER BY

```
SELECT  last_name, job_id, department_id,  hire_date
FROM employees
WHERE job_id = 'PU_CLERK'
ORDER BY hire_date
```

LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
Khoo	PU_CLERK	30	18/05/95
Tobias	PU_CLERK	30	24/07/97
Baida	PU_CLERK	30	24/12/97
Himuro	PU_CLERK	30	15/11/98
Colmenares	PU_CLERK	30	10/08/99

TRI DECROISSANT

```
SELECT  last_name, job_id, department_id,  hire_date
FROM employees
WHERE job_id = 'PU_CLERK'
ORDER BY hire_date desc
```

LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
Colmenares	PU_CLERK	30	10/08/99
Himuro	PU_CLERK	30	15/11/98
Baida	PU_CLERK	30	24/12/97
Tobias	PU_CLERK	30	24/07/97
Khoo	PU_CLERK	30	18/05/95

TRI SUR L'ALIAS DE COLONNE

```
SELECT employee_id, last_name, department_id,
salary * 12 "Salaire Annuel"
FROM employees
WHERE department_id = 30
ORDER BY "Salaire Annuel" DESC
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	Salaire Annuel
114	Raphaely	30	132000
115	Khoo	30	37200
116	Baida	30	34800
117	Tobias	30	33600
118	Himuro	30	31200
119	Colmenares	30	30000

TRI SUR LA POSITION DE LA COLONNE

```
SELECT employee_id, last_name, department_id,
salary * 12 "Salaire Annuel"
FROM employees
WHERE department_id = 30
ORDER BY 4 DESC
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	Salaire Annuel
114	Raphaely	30	132000
115	Khoo	30	37200
116	Baida	30	34800
117	Tobias	30	33600
118	Himuro	30	31200
119	Colmenares	30	30000

TRI SUR PLUSIEURS COLONNES

```
SELECT employee_id, last_name, department_id,
salary * 12 "Salaire Annuel"
FROM employees
where department_id <50
ORDER BY department_id , "Salaire Annuel" DESC
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	Salaire Annuel
200	Whalen	10	52800
201	Hartstein	20	156000
202	Fay	20	72000
114	Raphaely	30	132000
115	Khoo	30	37200
116	Baida	30	34800
117	Tobias	30	33600
118	Himuro	30	31200
119	Colmenares	30	30000
203	Mavris	40	78000

II. FONCTIONS MONO-LIGNES

A. Types de fonctions

B. Fonctions caractères

C. Fonctions numériques

D. Fonctions dates

E. Fonctions de conversion

F. Autres Fonctions

A. TYPES DE FONCTIONS

1. Fonctions mono-lignes : agissent sur une seule ligne et ramènent un seul résultat
2. Fonctions multi-lignes : manipulent des groupes de lignes et ramènent un seul résultat

B. FONCTIONS CARACTERES

1. Fonctions de conversion majuscules / minuscules
2. Fonctions de manipulation de caractères

1.FONCTIONS DE CONVERSION MAJUSCULES / MINUSCULES

- **LOWER** : convertit les caractères majuscules en minuscules
- **UPPER** : convertit les caractères minuscules en majuscules
- **INITCAP** : convertit l'initiale de chaque mot en majuscule et les caractères suivants en minuscules

```
SELECT  last_name, lower(last_name) , upper(last_name),
initcap(last_name)
FROM employees
```

LAST_NAME	LOWER(LAST_NAME)	UPPER(LAST_NAME)	INITCAP(LAST_NAME)
Abel	abel	ABEL	Abel
Ande	ande	ANDE	Ande
Atkinson	atkinson	ATKINSON	Atkinson
Austin	austin	AUSTIN	Austin
Baer	baer	BAER	Baer
Baida	baida	BAIDA	Baida
Banda	banda	BANDA	Banda
Bates	bates	BATES	Bates

2.FONCTIONS DE MANIPULATION DE CARACTERES

- **CONCAT** : concatène la première chaîne avec la seconde = ||
- **SUBSTR** : extrait une sous chaîne d'une autre chaîne
- **LENGTH** : taille d'une chaîne en caractères
- **INSTR** : position d'une chaîne de caractères dans une autre chaîne

```
Select employee_id, concat(concat(last_name,' '),first_name),
substr(last_name, 1,3), instr(last_name,'te'), length(last_name)
From employees
```

EMPLOYEE_ID	CONCAT(CONCAT(LAST_NAME,''),FIRST_NAME)	SUBSTR(LAST_NAME,1,3)	INSTR(LAST_NAME,'TE')	LENGTH(LAST_NAME)
100	King Steven	Kin	0	4
101	Kochhar Neena	Koc	0	7
102	De Haan Lex	De	0	7
103	Hunold Alexander	Hun	0	6

2.FONCTIONS DE MANIPULATION DE CARACTERES

- **TRIM** : élimine les espaces à gauche et à droite
- **LTRIM** : élimine les espaces à gauche
- **RTRIM** : élimine les espaces à droite

```
SELECT '  Foued OUELBANI  ' as "nom  prénom",
length('  Foued OUELBANI  ') as Taille, Ltrim('  Foued OUELBANI  ') as ltrim,
length(Ltrim('  Foued OUELBANI  ')) as  "taille ltrim",
rtrim('  Foued OUELBANI  ') as rtrim, length(rtrim('  Foued OUELBANI  '))
 as "taille rtrim",  trim('  Foued OUELBANI  ') as trim,
length( trim('  Foued OUELBANI  '))  as "taille rtrim"
FROM dual
```

Nom Prénom	TAILLE	LTRIM	Taille Ltrim	RTRIM	Taille Rtrim	TRIM	Taille Rtrim
Foued OUELBANI	21	Foued OUELBANI	18	Foued OUELBANI	17	Foued OUELBANI	14

2.FONCTIONS DE MANIPULATION DE CARACTERES

- **LPAD** : complète une chaîne de caractères sur la gauche avec une autre chaîne pour avoir n caractères.
- **RPAD** : complète une chaîne de caractères sur la droite avec une autre chaîne pour avoir n caractères.

```
SELECT  lpad('Foued OUELBANI' ,20, '+' ) as LPAD,
rpad('Foued OUELBANI' ,20, '+' ) as RPAD
FROM dual
```

LPAD	RPAD
+++++++Foued OUELBANI	Foued OUELBANI+++++++

2.FONCTIONS DE MANIPULATION DE CARACTERES

- **ASCII** : retourne le code ascii du premier caractère de la chaîne
- **CHR** : retourne le caractère (inverse de ascii)

SELECT ascii('FOUED OUELBANI '), chr(75) From dual

ASCII('FOUEDOUELBANI')	CHR(75)
70	K

2.FONCTIONS DE MANIPULATION DE CARACTERES

- **REPLACE** : remplace toutes les occurrences de la chaîne recherchée par une autre
- **TRANSLATE** : remplace chaque caractère recherché par son correspondant dans la chaîne principale

```
SELECT  'ABCDE ABDEU HRABDE ',
replace ('ABCDE ABDEU HRABDE ', 'AB','XYZ'),
translate('ABCDE ABDEU HRABDE ', 'AB','UVW'),
translate('ABCDE ABDEU HRABDE ', 'AB','U')
FROM dual
```

'ABCDEABDEUHRABDE'	REPLACE('ABCDEABDEUHRABDE','AB','XYZ')	TRANSLATE('ABCDEABDEUHRABDE','AB','UVW')	TRANSLATE('ABCDEABDEUHRABDE','AB','U')
ABCDE ABDEU HRABDE	XYZCDE XYZDEU HRXYZDE	UVCDE UVDEU HRUVDE	UCDE UDEU HRUDE

C. FONCTIONS NUMERIQUES

- **ROUND** : Arrondi la valeur à la précision spécifiée
- **TRUNC** : tronque la valeur à la précision spécifiée
- **FLOOR (x)** : si n<=x<n+1 alors FLOOR(x)=n
- **CEIL (x)** : si n<=x<n+1 alors CEIL(x)=n+1
- **MOD** : reste d'une division
- **REMAINDER (m, n)** : reste d'une division calculé comme suit :
  r=m-n*round(m/n,0)

C. FONCTIONS NUMERIQUES

```
SELECT ROUND(123.453,0),ROUND(123.553,0),ROUND(123.453,2),
ROUND(123.455,2),ROUND(123.453,-1), ROUND(163.453,-2)
FROM  dual
```

ROUND(123.453,0)	ROUND(123.553,0)	ROUND(123.453,2)	ROUND(123.455,2)	ROUND(123.453,-1)	ROUND(163.453,-2)
123	124	123,45	123,46	120	200

```
SELECT  trunc(123.453,0), trunc(123.453,1), trunc(123.453,2),
trunc(123.453,-1), trunc(163.453,-2)
FROM dual
```

TRUNC(123.453,0)	TRUNC(123.453,1)	TRUNC(123.453,2)	TRUNC(123.453,-1)	TRUNC(163.453,-2)
123	123,4	123,45	120	100

C. FONCTIONS NUMERIQUES

```
SELECT floor(-1.34), floor(-1.04), floor(-1.7),
       ceil(-1.34), ceil(-1.04), ceil(-1.7)
FROM dual
```

FLOOR(-1.34)	FLOOR(-1.04)	FLOOR(-1.7)	CEIL(-1.34)	CEIL(-1.04)	CEIL(-1.7)
-2	-2	-2	-1	-1	-1

C. FONCTIONS NUMERIQUES

```
SELECT mod(1260,430), mod(1260,430.5)
FROM dual
```

MOD(1260,430)	MOD(1260,430.5)
400	399

```
SELECT  remainder(15, 6),remainder(15.00001, 6),remainder(15, 5),
remainder(15, 4), remainder(11.6, 2), remainder(11.6, 2.1),
remainder(-15, 4)
FROM dual
```

REMAINDER(15,6)	REMAINDER(15.00001,6)	REMAINDER(15,5)	REMAINDER(15,4)	REMAINDER(11.6,2)	REMAINDER(11.6,2.1)	REMAINDER(-15,4)
3	-2,99999	0	-1	-,4	-1	1

D. FONCTIONS DATES

1. Ajout ou soustraction d'un nombre à une date
2. Soustraction de deux dates afin de déterminer le nombre de jours entre deux dates
3. Ajout d'un nombre d'heures à une date en divisant le nombre d'heures par 24

```
SELECT  sysdate, sysdate+2, to_date('20/09/1998') - 10,
trunc(sysdate - to_date('10/10/2008'),0)
FROM dual
```

SYSDATE	SYSDATE+2	TO_DATE('20/09/1998')-10	TRUNC(SYSDATE-TO_DATE('10/10/2008'),0)
08/01/19	10/01/19	10/09/98	3742

```
alter session set NLS_DATE_FORMAT='dd-mm-yyyy hh24:mi:ss';
```

D. FONCTIONS DATES

- **MONTHS_BETWEEN** : nombre de mois entre deux dates

```
SELECT sysdate,
months_between(sysdate,to_date('01/07/2018','dd-mm-yyyy')) ,
months_between('30/10/1990','20/10/1990')
FROM dual
```

SYSDATE	MONTHS_BETWEEN(SYSDATE,TO_DATE('01/07/2018','DD-MM-YYYY'))	MONTHS_BETWEEN('30/10/1990','20/10/1990')
28/01/19	6,897103121266427718040621266427718040 62	,322580645161290322580645161290322580645

- **ADD_MONTHS** : Ajoute des mois calendaires à une date

```
SELECT sysdate, add_months(sysdate,3)
FROM dual
```

SYSDATE	ADD_MONTHS(SYSDATE,3)
08/01/19	08/04/19

D. FONCTIONS DATES

- **NEXT_DAY : date du 1er jour semaine**
- **LAST_DAY : dernier jour du mois**

```
SELECT sysdate, next_day(sysdate, 'mardi'), last_day(sysdate)
FROM dual
```

SYSDATE	NEXT_DAY(SYSDATE,'SAMEDI')	LAST_DAY(SYSDATE)
08/01/19	12/01/19	31/01/19

D. FONCTIONS DATES

TO_DATE : convertit une chaine en une date selon un format

```
SELECT to_char(to_date('11-10-1981 17:25:15','dd-mm-yyyy
hh24:mi:ss'),'dd-mm-yyyy hh24:mi:ss') d1,
to_char(to_date('11-10-1981 17:25:15' ,'dd-mm-yyyy hh24:mi:ss'),
'dd-mm-yyyy hh24:mi:ss') d11,
to_date('11-10-1981 5:25:15' ,'dd-mm-yyyy hh12:mi:ss') d2,
to_char(to_date('11-10-1981 5:25:15','dd-mm-yyyy hh12:mi:ss'),
'dd-mm-yyyy hh24:mi:ss') d21
FROM dual
```

D1	D11	D2	D21
11-10-1981 17:25:15	11-10-1981 17:25:15	11/10/81	11-10-1981 05:25:15

D. FONCTIONS DATES

TO_CHAR : convertit une date en une chaine selon un format

```
SELECT employee_id Matricule, last_name Nom, first_name Prenom,
to_char(hire_date,'dd-mm-yyyy') "Date d'embauche",
to_char(hire_date,'dd') Jour, to_char(hire_date,'Day') "Jour Semaine",
to_char(hire_date,'mm') Mois, to_char(hire_date,'Mon') "Mois Abrégé",
to_char(hire_date,'MONTH') "Mois Complet",
to_char(hire_date,'w') "Semaine Mois",to_char(hire_date,'ww') "Semaine Année",
to_char(hire_date,'q') Trimestre, to_char(hire_date,'yyyy') Année
from employees
order by nom
```

MATRICULE	NOM	PRENOM	Date D'embauche	JOUR	Jour Semaine	MOIS	Mois Abrégé	Mois Complet	Semaine Mois	Semaine Année	TRIMESTRE	ANNÉE
174	Abel	Ellen	11-05-1996	11	Samedi	05	Mai	MAI	2	19	2	1996
166	Ande	Sundar	24-03-2000	24	Vendredi	03	Mars	MARS	4	12	1	2000
130	Atkinson	Mozhe	30-10-1997	30	Jeudi	10	Oct.	OCTOBRE	5	44	4	1997
105	Austin	David	25-06-1997	25	Mercredi	06	Juin	JUIN	4	26	2	1997
204	Baer	Hermann	07-06-1994	07	Mardi	06	Juin	JUIN	1	23	2	1994
116	Baida	Shelli	24-12-1997	24	Mercredi	12	Déc.	DÉCEMBRE	4	52	4	1997

D. FONCTIONS DATES

- **ROUND** : arrondi une date
- **TRUNC** : tronque une date

```
SELECT sysdate, months_between(sysdate,'01/07/2018') ,
round(months_between(sysdate,'01/07/2018'),0) Round0,
round(months_between(sysdate,'01/07/2018'),2) Round2,
trunc(months_between(sysdate,'01/07/2018'),0) Trunc0,
trunc(months_between(sysdate,'01/07/2018'),2) Trunc2
FROM dual
```

SYSDATE	MONTHS_BETWEEN(SYSDATE,'01/07/2018')	ROUND0	ROUND2	TRUNC0	TRUNC2
26/01/19	6,827048237753882915173237753882915173 24	7	6,83	6	6,82

D. FONCTIONS DATES

- **ROUND : arrondi une date : mn, hh, dd,w, ww,q, yyyy**

```
select to_char(sysdate,'dd-mm-yyyy hh24:mi:ss') "sysdate",
to_char(round(sysdate,'mi'),'dd-mm-yyyy hh24:mi:ss') round_mi,
to_char(round(sysdate, 'hh'),'dd-mm-yyyy hh24:mi:ss') round_hh,
to_char(round(sysdate,'dd'),'dd-mm-yyyy hh24:mi:ss') round_day,
to_char(round(sysdate,'w'),'dd-mm-yyyy hh24:mi:ss') round_w,
to_char(round(sysdate, 'ww'),'dd-mm-yyyy hh24:mi:ss') round_ww,
to_char(round(sysdate,'mm'),'dd-mm-yyyy hh24:mi:ss') round_m,
to_char(round(sysdate,'q'),'dd-mm-yyyy hh24:mi:ss') round_q,
to_char(round(sysdate, 'yy'),'dd-mm-yyyy hh24:mi:ss') round_yyyy
from dual
```

Sysdate	ROUND_MI	ROUND_HH	ROUND_DAY	ROUND_W	ROUND_WW	ROUND_M	ROUND_Q	ROUND_YYYY
26-01-2019 16:36:18	26-01-2019 16:36:00	26-01-2019 17:00:00	27-01-2019 00:00:00	29-01-2019 00:00:00	29-01-2019 00:00:00	01-02-2019 00:00:00	01-01-2019 00:00:00	01-01-2019 00:00:00

D. FONCTIONS DATES

- TRUNC : arrondi une date : mi, hh, dd,w, ww,q, yyyy

```
select to_char(sysdate,'dd-mm-yyyy hh24:mi:ss') "sysdate",
to_char(round(sysdate,'mi'),'dd-mm-yyyy hh24:mi:ss') round_mi,
to_char(round(sysdate, 'hh'),'dd-mm-yyyy hh24:mi:ss')
round_hh,to_char(round(sysdate,'dd'),'dd-mm-yyyy hh24:mi:ss')
round_day, to_char(round(sysdate,'w'),'dd-mm-yyyy hh24:mi:ss')
round_w, to_char(round(sysdate, 'ww'),'dd-mm-yyyy hh24:mi:ss')
round_ww, to_char(round(sysdate,'mm'),'dd-mm-yyyy hh24:mi:ss')
round_m, to_char(round(sysdate,'q'),'dd-mm-yyyy hh24:mi:ss')
round_q, to_char(round(sysdate, 'yy'),'dd-mm-yyyy hh24:mi:ss')
round_yyyy from dual
```

Sysdate	TRUNC_MI	TRUNC_HH	TRUNC_DAY	TRUNC_W	TRUNC_WW	TRUNC_M	TRUNC_Q	TRUNC_YYYY
26-01-2019 17:01:24	26-01-2019 17:01:00	26-01-2019 17:00:00	26-01-2019 00:00:00	22-01-2019 00:00:00	22-01-2019 00:00:00	01-01-2019 00:00:00	01-01-2019 00:00:00	01-01-2019 00:00:00

D. FONCTIONS DATES

```
SELECT systimestamp,
       extract(day  from  systimestamp)  as  Jour,
       extract(month  from  systimestamp) as  Mois,
       extract(year  from  systimestamp) as Année,
       extract(hour  from systimestamp)+2   as Hour,
       extract(minute from systimestamp) as Minute,
       trunc(extract(second from systimestamp))  as  Second
FROM dual
```

SYSTIMESTAMP	JOUR	MOIS	ANNÉE	HOUR	MINUTE	SECOND
08-JANV.-19 08.42.24,908000 PM +01:00	8	1	2019	21	42	24

F.AUTRES FONCTIONS

1. NVL : remplace une valeur nulle
   NVL2 (expr,val1,val2) : si expr n'est pas nulle elle est remplacée val1 sinon par val2
2. NULLIF (val1,val2) : si val1= val2 la valeur NULL est retournée sinon val1
3. Coalesce (exp1,expr2,expr3,…) : retourne la première valeur non nulle
4. Decode (expr, val1, val11, val2, val21, …. Valn, valn1, default) :
   retourne valn1 si expr = valn sinon default
5. Case : évalue une liste de conditions et retourne un résultat parmi les cas possibles
6. ROW_NUMBER : retourne le numéro séquentiel d'une ligne d'une partition d'un ensemble de résultats, en commençant à 1 pour la première ligne de chaque partition
7. Rank : retourne le rang de chaque ligne au sein de la partition d'un ensemble de résultats
8. DENSE_RANK : retourne le rang des lignes à l'intérieur de la partition d'un ensemble de résultats, sans aucun vide dans le classement
9. First_value : retourne la première valeur d'une partition
10. Last_value : retourne la dernière valeur d'une partition

1. NVL, NVL2

```
SELECT employee_id, last_name, salary, commission_pct, NVL(commission_pct,0),
    NVL(to_char(commission_pct), 'Pas de commission'),
    NVL2(commission_pct, commission_pct*100/salary,0),
    to_char(NVL2(commission_pct, commission_pct*100/salary,0)) || '%'
FROM employees
```

EMPNO	ENAME	SAL	COMM	NVL(COMM,0)	NVL(TO_CHAR(COMM),'PASDECOMMISSION')	NVL2(COMM,COMM*100/SAL,0)	TO_CHAR(NVL2(COMM,COMM*100/SAL,0))||'%'
7876	ADAMS	1100	-	0	Pas de commission	0	0%
7499	ALLEN	1600	300	300	300	18,75	18,75%
7698	BLAKE	2850	-	0	Pas de commission	0	0%
7782	CLARK	2450	-	0	Pas de commission	0	0%
7902	FORD	3000	-	0	Pas de commission	0	0%
7900	JAMES	950	-	0	Pas de commission	0	0%
7566	JONES	2975	-	0	Pas de commission	0	0%
7839	KING	5000	-	0	Pas de commission	0	0%
7654	MARTIN	1250	1400	1400	1400	112	112%
7934	MILLER	1300	-	0	Pas de commission	0	0%
7788	SCOTT	3000	-	0	Pas de commission	0	0%
7369	SMITH	800	-	0	Pas de commission	0	0%
7844	TURNER	1500	0	0	0	0	0%
7521	WARD	1250	500	500	500	40	40%

2. NULLIF

SELECT salary / commission_pct FROM employees

SELECT NULLIF(last_name, last_name), NULLIF(100, 200), salary, commission_pct, TRUNC(salary / NULLIF(commission_pct, 0),2) FROM employees

NULLIF(LAST_NAME,LAST_NAME)	NULLIF(100,200)	SALARY	COMMISSION_PCT	TRUNC(SALARY/NULLIF(COMMISSION_PCT,0),2)
-	100	24000	-	-
-	100	17000	-	-
-	100	17000	-	-
-	100	9000	-	-
-	100	6000	-	-
-	100	4800	-	-
-	100	4800	-	-
-	100	4200	-	-
-	100	12000	-	-
-	100	9000	-	-
-	100	8200	-	-
-	100	7700	-	-
-	100	7800	-	-

3. COALESCE

```
SELECT  coalesce(Null,5) from dual
```

COALESCE(NULL,5)
5

4. DECODE

```
SELECT employee_id,last_name,job_id, department_id,
decode(department_id,10, 'ACCOUNTING', 20, 'RESEARCH',
'DEP. INCONNU ')
FROM  employees ORDER BY department_id
```

EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID	DECODE(DEPARTMENT_ID,10,'ACCOUNTING',20,'RESEARCH','DEP.INCONNU')
200	Whalen	AD_ASST	10	ACCOUNTING
201	Hartstein	MK_MAN	20	RESEARCH
202	Fay	MK_REP	20	RESEARCH
114	Raphaely	PU_MAN	30	DEP. INCONNU
115	Khoo	PU_CLERK	30	DEP. INCONNU
116	Baida	PU_CLERK	30	DEP. INCONNU
117	Tobias	PU_CLERK	30	DEP. INCONNU
118	Himuro	PU_CLERK	30	DEP. INCONNU
119	Colmenares	PU_CLERK	30	DEP. INCONNU

5. CASE

```
SELECT employee_id, last_name, department_id,
    case department_id
        when 10 then 'Administration'
        when 20 then 'Marketing'
        when 30 then 'Purchasing'
        else 'Département Erroné'
   end as departement
FROM employees
ORDER BY 3
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTEMENT
200	Whalen	10	Administration
201	Hartstein	20	Marketing
202	Fay	20	Marketing
114	Raphaely	30	Purchasing
115	Khoo	30	Purchasing
116	Baida	30	Purchasing
117	Tobias	30	Purchasing
118	Himuro	30	Purchasing
119	Colmenares	30	Purchasing
203	Mavris	40	Département Erroné
120	Weiss	50	Département Erroné

5. CASE

```
SELECT employee_id, last_name, department_id,
case
   when department_id < 10 or department_id >30 then 999
   else department_id
end as departement
FROM employees
ORDER BY  3
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTEMENT
200	Whalen	10	10
201	Hartstein	20	20
202	Fay	20	20
114	Raphaely	30	30
115	Khoo	30	30
116	Baida	30	30
117	Tobias	30	30
118	Himuro	30	30
119	Colmenares	30	30
203	Mavris	40	999
120	Weiss	50	999

6. ROW_NUMBER

```
SELECT department_id, employee_id, salary,
row_number() over(order by salary desc) NO
FROM employees
Order by no
```

DEPARTMENT_ID	EMPLOYEE_ID	SALARY	NO
90	100	24000	1
90	101	17000	2
90	102	17000	3
80	145	14000	4
80	146	13500	5
20	201	13000	6
100	108	12000	7
80	147	12000	8
110	205	12000	9
80	168	11500	10

```
SELECT department_id, employee_id, salary,
row_number() over(partition by department_id
order by salary desc) NO
FROM employees
Order by department_id,no
```

DEPARTMENT_ID	EMPLOYEE_ID	SALARY	NO
10	200	4400	1
20	201	13000	1
20	202	6000	2
30	114	11000	1
30	115	3100	2
30	116	2900	3
30	117	2800	4
30	118	2600	5
30	119	2500	6
40	203	6500	1
50	121	8200	1
50	120	8000	2
50	122	7900	3

7. RANK

```
SELECT department_id, employee_id, salary,
rank() over(order by salary desc) Rank
FROM employees
Order by 4
```

DEPARTMENT_ID	EMPLOYEE_ID	SALARY	RANK
90	100	24000	1
90	101	17000	2
90	102	17000	2
80	145	14000	4
80	146	13500	5
20	201	13000	6
100	108	12000	7
80	147	12000	7
110	205	12000	7
80	168	11500	10

```
SELECT department_id, employee_id, salary,
rank() over(partition by department_id
order by salary desc) Rank
FROM employees
Order by department_id,rank
```

DEPARTMENT_ID	EMPLOYEE_ID	SALARY	RANK
10	200	4400	1
20	201	13000	1
20	202	6000	2
30	114	11000	1
30	115	3100	2
30	116	2900	3
30	117	2800	4
30	118	2600	5
30	119	2500	6
40	203	6500	1
50	121	8200	1
50	120	8000	2

8. DENSE_RANK

```
SELECT department_id, employee_id, salary,
dense_rank() over(order by salary desc) Rank
FROM employees
Order by 4
```

DEPARTMENT_ID	EMPLOYEE_ID	SALARY	RANK
90	100	24000	1
90	101	17000	2
90	102	17000	2
80	145	14000	3
80	146	13500	4
20	201	13000	5
100	108	12000	6
80	147	12000	6
110	205	12000	6
80	168	11500	7
80	174	11000	8
80	148	11000	8
30	114	11000	8
80	149	10500	9
80	162	10500	9
80	156	10000	10

```
SELECT department_id, employee_id, salary,
dense_rank() over(partition by department_id
order by salary desc) Rank
FROM employees
Order by department_id,rank
```

DEPARTMENT_ID	EMPLOYEE_ID	SALARY	RANK
10	200	4400	1
20	201	13000	1
20	202	6000	2
30	114	11000	1
30	115	3100	2
30	116	2900	3
30	117	2800	4
30	118	2600	5
30	119	2500	6
40	203	6500	1
50	121	8200	1
50	120	8000	2
50	122	7900	3

9. FIRST_VALUE

```
SELECT employee_id, salary,last_name,
First_value(last_name) over(order by salary desc, last_name desc)
First_value
FROM employees
where salary between 2500 and 2800
Order by 2 desc,3 desc
```

EMPLOYEE_ID	SALARY	LAST_NAME	FIRST_VALUE
117	2800	Tobias	Tobias
195	2800	Jones	Tobias
183	2800	Geoni	Tobias
130	2800	Atkinson	Tobias
139	2700	Seo	Tobias
126	2700	Mikkilineni	Tobias
198	2600	OConnell	Tobias
143	2600	Matos	Tobias
118	2600	Himuro	Tobias
199	2600	Grant	Tobias

```
SELECT department_id, employee_id, last_name,salary,
First_value(last_name) over(partition by department_id
order by salary desc) First_value
FROM employees
Order by department_id, salary
```

DEPARTMENT_ID	EMPLOYEE_ID	LAST_NAME	SALARY	FIRST_VALUE
10	200	Whalen	4400	Whalen
20	202	Fay	6000	Hartstein
20	201	Hartstein	13000	Hartstein
30	119	Colmenares	2500	Raphaely
30	118	Himuro	2600	Raphaely
30	117	Tobias	2800	Raphaely
30	116	Baida	2900	Raphaely
30	115	Khoo	3100	Raphaely
30	114	Raphaely	11000	Raphaely

10. LAST_VALUE

```
SELECT employee_id, salary,last_name,
last_value(last_name) over(order by salary desc, last_name desc)
last_value
FROM employees
where salary between 2500 and 2800
Order by 2 desc,3 desc

SELECT department_id, employee_id, last_name,salary,
last_value(last_name) over(partition by department_id
order by salary desc) last_value
FROM employees
Order by department_id, salary
```

III. FONCTIONS DE GROUPES

Les fonctions de groupe agissent sur des groupes de lignes et donnent un résultat par groupe.

AVG([distinct|all]expr) : valeur moyenne en ignorant les valeurs NULL

COUNT ([*|distinct|all]expr) : nombre de lignes où expr est différente de NULL. Le caractère * comptabilise toutes les lignes sélectionnées.

MAX ([distinct|all]expr) : valeur maximale en en ignorant les valeurs NULL

MIN([distinct|all]expr) : valeur minimale en en ignorant les valeurs NULL

STDDEV([distinct|all]expr) : ecart-type en en ignorant les valeurs NULL

SUM([distinct|all]expr) : somme en en ignorant les valeurs NULL

VARIANCE([distinct|all]expr) : variance en ignorant les valeurs NULL

FONCTIONS DE GROUPES

```
SELECT count(salary) as count, trunc(sum(salary),3) as sum,
       min (salary) as min, max(salary) as max,
       trunc(avg(salary),3) as avg,
       trunc(variance(salary),3) as variance,
       trunc(stddev(salary),3)  as ecart
FROM employees
```

COUNT	SUM	MIN	MAX	AVG	VARIANCE	ECART
107	691400	2100	24000	6461,682	15283140,539	3909,365

```
SELECT max(last_name), min(last_name), count(last_name)
From employees
```

MAX(LAST_NAME)	MIN(LAST_NAME)	COUNT(LAST_NAME)
Zlotkey	Abel	107

```
SELECT max(hire_date), min(hire_date), count(hire_date)
FROM employees
```

MAX(HIRE_DATE)	MIN(HIRE_DATE)	COUNT(HIRE_DATE)
21/04/00	17/06/87	107

FONCTIONS DE GROUPES
CLAUSE GROUP BY

```
SELECT department_id deptno, max(last_name), min(last_name),
count(last_name)
FROM employees
GROUP BY department_id
ORDER BY department_id
```

DEPTNO	MAX(LAST_NAME)	MIN(LAST_NAME)	COUNT(LAST_NAME)
10	Whalen	Whalen	1
20	Hartstein	Fay	2
30	Tobias	Baida	6
40	Mavris	Mavris	1
50	Weiss	Atkinson	45
60	Pataballa	Austin	5
70	Baer	Baer	1
80	Zlotkey	Abel	34
90	Kochhar	De Haan	3
100	Urman	Chen	6
110	Higgins	Gietz	2
-	Grant	Grant	1

```
SELECT department_id deptno, max(hire_date), min(hire_date),
count(hire_date)
FROM employees
GROUP BY department_id
ORDER BY department_id
```

DEPTNO	MAX(HIRE_DATE)	MIN(HIRE_DATE)	COUNT(HIRE_DATE)
10	17/09/87	17/09/87	1
20	17/08/97	17/02/96	2
30	10/08/99	07/12/94	6
40	07/06/94	07/06/94	1
50	08/03/00	01/05/95	45
60	07/02/99	03/01/90	5
70	07/06/94	07/06/94	1
80	21/04/00	30/01/96	34
90	13/01/93	17/06/87	3
100	07/12/99	16/08/94	6
110	07/06/94	07/06/94	2
-	24/05/99	24/05/99	1

FONCTIONS DE GROUPES
CLAUSE HAVING

```
SELECT department_id  deptno, count(*) as "nb salariés",
sum(salary)  "Total salaire"
FROM employees
GROUP BY department_id
Having count(*) >3 and sum(salary) >10000
ORDER BY department_id
```

DEPTNO	Nb Salariés	Total Salaire
30	6	24900
50	45	156400
60	5	28800
80	34	304500
100	6	51600

```
SELECT department_id deptno, max(last_name),
min(last_name), count(last_name)
FROM employees
GROUP BY department_id
HAVING max(last_name) <'P'
and min(last_name) <'M'
ORDER BY department_id
```

DEPTNO	MAX(LAST_NAME)	MIN(LAST_NAME)	COUNT(LAST_NAME)
20	Hartstein	Fay	2
70	Baer	Baer	1
90	Kochhar	De Haan	3
110	Higgins	Gietz	2
-	Grant	Grant	1

FONCTIONS DE GROUPES
CLAUSE HAVING

```
SELECT department_id deptno, max(hire_date),
min(hire_date), count(hire_date)
FROM employees
GROUP BY department_id
HAVING max(hire_date) > '01-01-1996'  and count(hire_date) >5
ORDER BY department_id
```

DEPTNO	MAX(HIRE_DATE)	MIN(HIRE_DATE)	COUNT(HIRE_DATE)
30	10/08/99	07/12/94	6
50	08/03/00	01/05/95	45
80	21/04/00	30/01/96	34
100	07/12/99	16/08/94	6

IV.JOINTURES

1. Définition d'une jointure
2. Produit cartésien
3. Jointure naturelle
4. Jointure interne
5. Jointure externe
6. Auto-jointure
7. Non-équijointure

1. DEFINITION D'UNE JOINTURE

- Une jointure sert à extraire des données de 2 tables
- La condition de jointure est exprimée dans la clause ON :
    - On T1.C1=T2.C1
- Précédez le nom de la colonne par le nom de la table lorsque celui-ci figure dans plusieurs tables

2. PRODUIT CARTESIEN

- On obtient un produit cartésien lorsque :
  - Une condition de jointure est omise
  - Une condition de jointure est incorrecte
- A chaque ligne de la table T1 sont jointes toutes les lignes de la table T2.
- Le nombre de lignes renvoyés est égal n1*n2 où n1 est le nombre de lignes de la table T1 et n2 est le nombre de lignes de la table T2.

Exemple

SELECT * FROM T1,T2

3. JOINTURE NATURELLE

SELECT * FROM T1 NATURAL JOIN T2
WHERE CONDITION(S)

SELECT department_id, department_name,
employee_id empno, last_name nom, salary salaire
From employees natural join departments
Order by department_id, employee_id

DEPARTMENT_ID	DEPARTMENT_NAME	EMPNO	NOM	SALAIRE
20	Marketing	202	Fay	6000
30	Purchasing	115	Khoo	3100
30	Purchasing	116	Baida	2900
30	Purchasing	117	Tobias	2800
30	Purchasing	118	Himuro	2600
30	Purchasing	119	Colmenares	2500
50	Shipping	129	Bissot	3300

NB : Noter le nombre de lignes retournées
La jointure a été faite sur quelle(s) colonne(s)

4. JOINTURE INTERNE

```
SELECT T1.co11, … T1.coln, T2.col2, …T2.colm
FROM T1  INNER JOIN T2
ON  T1.Coli=T2.Colj
WHERE CONDITION(S)
```

```
SELECT employee_id empno, last_name nom, salary salaire,
employees.department_id deptno, department_name departement
FROM  employees   INNER JOIN departments
ON  employees.department_id=departments.department_id
order by deptno, nom
```

EMPNO	NOM	SALAIRE	DEPTNO	DEPARTEMENT
200	Whalen	4400	10	Administration
202	Fay	6000	20	Marketing
201	Hartstein	13000	20	Marketing
116	Baida	2900	30	Purchasing
119	Colmenares	2500	30	Purchasing
118	Himuro	2600	30	Purchasing
115	Khoo	3100	30	Purchasing

5. JOINTURE EXTERNE

```
SELECT T1.Col1,  …T1.Coln, T2.Col1,  …T2.Colm
FROM T1  LEFT [OUTER] JOIN T2
ON  T1.Coli=T2.Colj
```

```
SELECT T1.Col1,  …T1.Coln, T2.Col1,  …T2.Colm
FROM T1 RIGHT [OUTER] JOIN T2
ON  T1.Coli=T2.Colj
```

```
SELECT T1.Col1,  …T1.Coln, T2.Col1,  …T2.Colm
FROM T1 FULL [OUTER] JOIN T2
ON  T1.Coli=T2.Colj
```

6. AUTO-JOINTURE

Liaison d'une table à elle-même

```
SELECT a.employee_id Matricule, a.last_name Nom, a.job_id
  "Code grade", b.last_name "Nom Responsable"
FROM employees a left Join employees b
on a.manager_id=b.employee_id
ORDER BY a.employee_id
```

MATRICULE	NOM	Code Grade	Nom Responsable
100	King	AD_PRES	-
101	Kochhar	AD_VP	King
102	De Haan	AD_VP	King
103	Hunold	IT_PROG	De Haan

7. NON-EQUIJOINTURE

```
SELECT employee_id Matricule, last_name Nom, first_name
  prénom, salary Salaire, notranche,salaire_min, salaire_max
FROM employees a left Join tranchesalaires b
on salary >=salaire_min and salary <salaire_max
ORDER BY  last_name
```

NOTRANCHE	TRANCHE	SALAIRE_MIN	SALAIRE_MAX
1	[0-5000[	0	5000
2	[5000-10000[	5000	10000
3	[10000-15000[	10000	15000
4	[15000-20000[	15000	20000
5	[20000-50000[	20000	50000

MATRICULE	NOM	PRÉNOM	SALAIRE	NOTRANCHE	SALAIRE_MIN	SALAIRE_MAX
174	Abel	Ellen	11000	3	10000	15000
166	Ande	Sundar	6400	2	5000	10000
130	Atkinson	Mozhe	2800	1	0	5000
105	Austin	David	4800	1	0	5000
204	Baer	Hermann	10000	3	10000	15000
116	Baida	Shelli	2900	1	0	5000

IV. OPERATEURS ENSEMBLISTES

OPERATEUR	DESCRIPTION
INTERSECT	Ramène toutes les lignes communes aux deux requêtes
UNION	Toutes les lignes distinctes ramenées par les deux requêtes
UNION ALL	Toutes les lignes ramenées par les deux requêtes y compris les doublons
MINUS	Toutes les lignes ramenées par la première requête sauf les lignes ramenées par la seconde requête

1. UNION

Le nombre de colonnes et le type des colonnes doivent être identiques dans les 2 ordres Select
L'opérateur UNION intervient sur toutes les colonnes

```
SELECT employee_id, job_id, salary
FROM employees
WHERE  job_id like 'A%'
ORDER BY 1
```

EMPLOYEE_ID	JOB_ID	SALARY
100	AD_PRES	24000
101	AD_VP	17000
102	AD_VP	17000
200	AD_ASST	4400
205	AC_MGR	12000
206	AC_ACCOUNT	8300

```
SELECT  employee_id, job_id, salary
from employees
where  salary >= 12000
ORDER BY 1
```

EMPLOYEE_ID	JOB_ID	SALARY
100	AD_PRES	24000
101	AD_VP	17000
102	AD_VP	17000
108	FI_MGR	12000
145	SA_MAN	14000
146	SA_MAN	13500
147	SA_MAN	12000
201	MK_MAN	13000
205	AC_MGR	12000

1. UNION

```
SELECT employee_id, job_id, salary
FROM employees
WHERE  job_id like 'A%'
        UNION
SELECT  employee_id, job_id, salary
from employees
where  salary >= 12000
ORDER BY 1
```

EMPLOYEE_ID	JOB_ID	SALARY
100	AD_PRES	24000
101	AD_VP	17000
102	AD_VP	17000
108	FI_MGR	12000
145	SA_MAN	14000
146	SA_MAN	13500
147	SA_MAN	12000
200	AD_ASST	4400
201	MK_MAN	13000
205	AC_MGR	12000
206	AC_ACCOUNT	8300

1. UNION ALL

Les doublons ne sont pas éliminés

```
SELECT employee_id, job_id, salary
FROM employees
WHERE  job_id like 'A%'
        UNION ALL
SELECT  employee_id, job_id, salary
from employees
where  salary >= 12000
ORDER BY 1
```

EMPLOYEE_ID	JOB_ID	SALARY
100	AD_PRES	24000
100	AD_PRES	24000
101	AD_VP	17000
101	AD_VP	17000
102	AD_VP	17000
102	AD_VP	17000
108	FI_MGR	12000
145	SA_MAN	14000
146	SA_MAN	13500
147	SA_MAN	12000
200	AD_ASST	4400
201	MK_MAN	13000
205	AC_MGR	12000
205	AC_MGR	12000
206	AC_ACCOUNT	8300

2. INTERSECTION

```
SELECT employee_id, job_id, salary
FROM employees
WHERE  job_id like 'A%'
Intersect
SELECT  employee_id, job_id, salary
from employees
where  salary >= 12000
ORDER BY 1
```

EMPLOYEE_ID	JOB_ID	SALARY
100	AD_PRES	24000
101	AD_VP	17000
102	AD_VP	17000
205	AC_MGR	12000

3. MINUS

```
SELECT employee_id, job_id, salary
FROM employees
WHERE  job_id like 'A%'
Minus
SELECT  employee_id, job_id, salary
from employees
where  salary >= 12000
ORDER BY 1
```

EMPLOYEE_ID	JOB_ID	SALARY
200	AD_ASST	4400
206	AC_ACCOUNT	8300

```
SELECT  employee_id, job_id, salary
from employees
where  salary >= 12000
Minus
SELECT employee_id, job_id, salary
FROM employees
WHERE  job_id like 'A%'
ORDER BY 1
```

EMPLOYEE_ID	JOB_ID	SALARY
108	FI_MGR	12000
145	SA_MAN	14000
146	SA_MAN	13500
147	SA_MAN	12000
201	MK_MAN	13000

V. SOUS-INTERROGATIONS

```
SELECT select_liste
FROM table
WHERE expr operateur (SELECT select_liste  FROM table)
```

- La sous-interrogation (requête interne) est exécutée une seule fois avant la requête principale
- Le résultat de la sous-interrogation est utilisé par la requête principale (requête externe)
- Une sous-interrogation est utilisée dans les clauses suivantes :
  - WHERE
  - HAVING
  - FROM
- Les opérateurs de comparaison mono-ligne (>,>= ,<, <=, …)
- Les opérateurs de comparaison multi-ligne (IN, ALL, ANY)

C. SOUS-INTERROGATIONS MONO-LIGNE

```
SELECT  ename, job, sal FROM emp
WHERE job =(SELECT  job FROM  emp WHERE  ename ='FORD' )

SELECT ename, job, sal  FROM emp
WHERE sal =(SELECT MIN(sal) FROM emp)

SELECT deptno, min(sal) FROM emp
GROUP BY deptno
HAVING min(sal)>(SELECT min(sal) FROM emp WHERE deptno=20)
```

C. SOUS-INTERROGATIONS MULTI-LIGNES

```
SELECT ename,sal,deptno FROM emp
WHERE sal IN (SELECT min(sal)
FROM emp GROUP BY deptno)
```

ENAME	SAL	DEPTNO
JAMES	950	30
SMITH	800	20
MILLER	1300	10

```
SELECT ename,sal,deptno FROM emp
WHERE sal < ANY (SELECT min(sal)
FROM emp group by deptno)
```

ENAME	SAL	DEPTNO
SMITH	800	20
JAMES	950	30
ADAMS	1100	20
MARTIN	1250	30
WARD	1250	30

```
SELECT ename,sal,deptno FROM emp
WHERE sal > ALL (SELECT min(sal)
FROM emp group by deptno)
```

ENAME	SAL	DEPTNO
KING	5000	10
BLAKE	2850	30
CLARK	2450	10
JONES	2975	20
SCOTT	3000	20
FORD	3000	20
ALLEN	1600	30
TURNER	1500	30

C. SOUS-INTERROGATIONS EXISTS

```
SELECT department_id FROM departments d
WHERE EXISTS   (SELECT * FROM employees e
 WHERE d.department_id =  e.department_id)
```

DEPARTMENT_ID
10
20
30
40
50
60
70
80
90
100
110

```
SELECT department_id FROM departments d WHERE Not EXISTS
(SELECT * FROM employees e  WHERE d.department_id
    = e.department_id)
```

DEPARTMENT_ID
120
130
140
150
160
170
180
190
200
210
220
230
240
250
260
270